



**UNIVERSIDAD
DE ANTIOQUIA**

Facultad de Ingeniería

ProcGuard

Sistema de Gobernanza Activa de Procesos para Linux

Monitoreo multidimensional • Detección de anomalías • Gobernanza automatizada

Detección de comportamiento sospechoso • Captura forense • Reportes HTML

INTEGRANTES

Brahian Ocampo Garcia

CURSO

Sistemas Operativos

Ronal Montoya

2026-1

CONTROL DE VERSIONES DEL DOCUMENTO

Versión	Fecha	Descripción cambio	Elaborado por	Aprobado por
1.0	27/03/2026	Definición de propuesta del proyecto	Brahian Ocampo	
2.0	27/04/2026	Ajuste y redefinición final del proyecto	Brahian Ocampo	

Tabla de Contenido

Tabla de Contenido	2
1. Resumen	3
2. Introducción	3
2.1. Contexto y motivación	3
2.2. Planteamiento del problema	4
2.3. Propuesta de solución	5
3. Objetivos	5
3.1. Objetivo general	5
3.2. Objetivos específicos	5
4. Marco Teórico	6
4.1. Pseudo-sistema de archivos /proc (procf)	6
4.2. Cálculo de métricas derivadas	7
4.3. Mecanismos de control de recursos del kernel	8
4.4. Detección de comportamiento sospechoso	9
5. Arquitectura del Sistema	9
5.1. Módulo 1: Recolector de datos (Data Collector)	9
5.2. Módulo 2: Almacén de muestras (Sample Store)	9
5.3. Módulo 3: Motor de métricas (Metrics Engine)	10
5.4. Módulo 4: Motor de alertas y gobernanza (Alert & Governance Engine)	10
5.5. Módulo 5: Motor de seguridad (Security Engine)	13
5.6. Módulo 6: Interfaz de terminal interactiva (TUI)	16
5.7. Módulo 7: Reportes y registro forense	17
5.8. Modos de Operación	18
5.9. Diagrama de Arquitectura	19
5.10. Ciclo de Gobernanza	20
5.11. Concurrencia	21
6. Manejo de Errores y Resiliencia	22
7. Protección del Sistema	23
8. Estrategia de Testing	24
9. Tecnologías y Herramientas	25
10. Diseño Experimental	25
10.1. Dimensión 1: Funcionalidad – Precisión de detección y efectividad de gobernanza	26
10.2. Dimensión 2: Seguridad – Detección de comportamiento sospechoso	27
10.3. Protocolo experimental común	28
10.4. Resumen de métodos estadísticos	29
11. Cronograma	29

1. Resumen

ProcGuard es un sistema de gobernanza activa de procesos para Linux que trasciende el paradigma tradicional de monitoreo pasivo. A diferencia de herramientas como **top** o **htop**, que se limitan a mostrar métricas del sistema, ProcGuard implementa un ciclo completo de **observación** → **detección** → **decisión** → **acción** → **registro** sobre los procesos en ejecución, gobernado por políticas configurables definidas por el administrador del sistema.

El sistema se desarrolla íntegramente en lenguaje **C** sobre Linux, utilizando **procfs** como fuente primaria de información, **inotify** como mecanismo de notificación del sistema de archivos para detección de seguridad y **cgroups v2** como mecanismo de control de recursos. Su arquitectura contempla siete módulos o subsistemas integrados: recolector de métricas multidimensional (CPU, memoria, I/O, red e hilos), motor de cálculo de métricas derivadas, motor de detección de anomalías de rendimiento, motor de gobernanza activa con respuesta automatizada, motor de detección de comportamiento sospechoso orientado a seguridad, interfaz de terminal interactiva (TUI), y generador de reportes HTML con registro forense de incidentes.

El proyecto se diferencia de las herramientas existentes en cinco dimensiones: ejecución de acciones correctivas automáticas con protecciones contra daño autoinfligido; captura forense del estado completo del proceso en el momento de la alerta; detección de patrones de comportamiento sospechoso asociados a actividad maliciosa mediante heurísticas acumulativas y basadas en eventos (inotify); generación de reportes HTML autocontenidos; y un sistema de resiliencia en tres niveles que garantiza operación continua antes fallos parciales.

2. Introducción

2.1. Contexto y motivación

Los sistemas Linux modernos operan bajo condiciones de creciente complejidad: entornos multiusuario, cargas mixtas de CPU, memoria, I/O y red, servicios persistentes, contenedores y máquinas virtuales coexistiendo sobre el mismo hardware.

En este contexto, la observación pasiva resulta insuficiente. Un administrador que detecta un proceso problemático mediante top debe ejecutar manualmente una cadena de acciones: identificar el proceso, evaluar su impacto, decidir una acción correctiva y ejecutarla mediante comandos separados (*kill*, *renice*, *taskset*, *cgcreate*). Este flujo manual introduce latencia, riesgo de error humano y ausencia de trazabilidad.

Adicionalmente, la seguridad del sistema requiere vigilancia continua sobre el comportamiento de los procesos. Un proceso malicioso que escanea puertos, exfiltra datos o se camufla imitando nombres de servicios legítimos puede pasar desapercibido para herramientas de monitoreo convencionales que solo muestran métricas de rendimiento sin analizar patrones de comportamiento.

Linux ofrece a nivel del kernel mecanismos sofisticados de medición y control de recursos —*procfs*, *cgroups v2*, *sched_setaffinity*, *sched_setscheduler*— que permiten observar y actuar programáticamente sobre los procesos. Sin embargo, ninguna herramienta de terminal integra observación multidimensional, detección de anomalías de rendimiento, detección de comportamiento sospechoso, respuesta automatizada, protecciones de sistema y registro forense en un solo sistema.

2.2. Planteamiento del problema

Las herramientas actuales de monitoreo de procesos en Linux presentan las siguientes limitaciones agrupadas en tres ejes:

- **Observación limitada:** La observación es pasiva y desconectada de la acción correctiva. La información de red por proceso es ignorada o tratada como un módulo separado. No se genera un registro histórico estructurado y navegable de incidentes.
- **Detección limitada:** Las alertas, cuando existen, se basan en umbrales estáticos sin persistencia temporal ni mecanismos contra falsos positivos. No tienen capacidad de detección de patrones de comportamiento sospechoso o malicioso a nivel de proceso. No hay captura automática del estado del proceso en el momento de la anomalía.
- **Acción limitada:** La respuesta ante anomalías es manual, sin escalamiento progresivo ni trazabilidad de las acciones ejecutadas.

2.3. Propuesta de solución

ProcGuard aborda estas limitaciones mediante un sistema que integra políticas declarativas definidas por el administrador del sistema, las cuales especifican umbrales de rendimiento, heurísticas de seguridad, condiciones de activación, acciones correctivas y parámetros de notificación. El sistema ejecuta autónomamente estas políticas con protecciones multinivel contra daño autoinfligido, registra cada acción con evidencia forense y genera reportes HTML navegables para su posterior análisis.

3. Objetivos

3.1. Objetivo general

Diseñar e implementar un sistema de gobernanza activa de procesos para Linux que integre el ciclo completo de observación, detección, decisión, acción y registro sobre los procesos en ejecución, gobernado por políticas configurables definidas por el administrador del sistema, operando en modo interactivo (TUI) y como daemon.

3.2. Objetivos específicos

1. Implementar un recolector de métricas multidimensional basado en procfs que obtenga información de CPU, memoria, I/O y red, tanto a nivel global como por proceso, con manejo resiliente de errores en lectura.
2. Construir una interfaz de terminal interactiva (TUI) basada en ncurses que permita listar, ordenar, filtrar y visualizar procesos con paneles especializados y acciones interactivas.
3. Desarrollar un motor de detección de anomalías de rendimiento configurable, con persistencia temporal, histéresis para desactivación de alertas y período de enfriamiento para evitar falsos positivos.
4. Implementar un motor de gobernanza activa con acciones correctivas escalonadas (warn → renice → cage → stop → kill) y protecciones de sistema.

5. Desarrollar un motor de detección de comportamiento sospechoso que identifica patrones asociados a actividad maliciosa, basado en cuatro heurísticas: escaneo de puertos, procesos camuflados, enumeración del sistema y procesos huérfanos anómalos.
6. Implementar captura forense automática ante cada alerta de rendimiento o de seguridad.
7. Implementar la generación de reportes HTML autocontenidos mediante arquitectura template + JSON.
8. Implementar almacenamiento de métricas en log estructurado (JSON lines) con rotación automática por tamaño y por sesión.
9. Implementar modo daemon con gestión de señales POSIX (SIGUSR1, SIGUSR2, SIGHUP, SIGTERM) para operación autónoma en producción.
10. Diseñar y ejecutar un experimento controlado que evalúe la precisión de detección de anomalías de rendimiento, efectividad de gobernanza y capacidad de detección de comportamiento sospechoso, con análisis estadístico formal.

4. Marco Teórico

4.1. Pseudo-sistema de archivos /proc (procfs)

procfs es la interfaz primaria del kernel de Linux para exponer información del sistema y de procesos al espacio de usuario mediante archivos virtuales legibles. ProcGuard utiliza archivos globales del sistema y archivos por proceso como fuente de datos.

Archivos globales del sistema

Archivo	Información que expone	Uso en ProcGuard
/proc/stat	Tiempos acumulados de CPU por core	Cálculo de uso porcentual de CPU global y por core

/proc/meminfo	Estadísticas de memoria del sistema	Dashboard de memoria; detección de presión de memoria
/proc/loadavg	Carga media del sistema (1, 5, 15 min)	Indicador de carga en TUI
/proc/net/dev	Contadores de bytes por interfaz de red	Cálculo de tasas de tráfico por interfaz
/proc/net/tcp , /proc/net/udp	Tabla de sockets TCP/UDP	Mapeo de conexiones de red a procesos

Archivos por proceso

Archivo	Información que expone	Uso en ProcGuard
/proc/[pid]/stat	Estado, tiempos de CPU, prioridad, hilos, starttime	Cálculo de %CPU; identificación única (pid, starttime)
/proc/[pid]/status	VmRSS, VmSize, hilos, UID/GID real y efectivo	Medición de memoria; detección de cambio de privilegios
/proc/[pid]/io	Bytes leídos/escritos	Métricas de I/O
/proc/[pid]/fd/	Descriptores de archivo abiertos	Correlación socket→proceso
/proc/[pid]/maps	Regiones de memoria virtual	Captura forense
/proc/[pid]/exe	Enlace al ejecutable real del proceso	Detección de procesos camuflados

4.2. Cálculo de métricas derivadas

Uso de CPU

El uso de CPU por proceso no es un valor instantáneo sino la tasa derivada entre dos muestras consecutivas, comparando la variación de tiempos de CPU del proceso (utime + stime) respecto al cambio total de tiempo de CPU del sistema entre dos instantes de muestreo. Para garantizar robustez frente a ajustes del reloj del sistema, se utiliza clock_gettime() con CLOCK_MONOTONIC.

Medición de memoria

La medición de memoria utiliza VmRSS (Resident Set Size) obtenido de /proc/[pid]/status, que representa la memoria física actualmente asignada al proceso.

Esta métrica tiene bajo costo de lectura y es suficiente para la detección de anomalías de consumo de memoria y para las acciones de gobernanza. VmSize (memoria virtual total) se registra como métrica complementaria para contexto en los snapshots forenses.

Monitoreo de red por proceso

Linux no expone el tráfico de red por proceso directamente. ProcGuard resuelve esto mediante correlación indirecta: lee las tablas de sockets desde `/proc/net/tcp` y `/proc/net/udp`, recorre `/proc/[pid]/fd/` identificando sockets mediante `readlink()` y correlaciona inodos para mapear qué conexiones pertenecen a cada proceso. Este mecanismo alimenta la estructura acumulativa para detección de escaneo de puertos.

Dado el costo computacional de recorrer `/proc/[pid]/fd/` para todos los procesos, la correlación de red no se ejecuta en cada ciclo de muestreo sino cada N ciclos, controlado por el parámetro configurable `net_sample_divisor` (valor por defecto: 4). Las demás métricas (CPU, memoria, I/O) se actualizan en cada ciclo.

4.3. Mecanismos de control de recursos del kernel

ProcGuard utiliza cuatro mecanismos de control de recursos del kernel, que constituyen la base de las acciones de gobernanza.

- **Señales POSIX:** SIGSTOP (suspensión inmediata), SIGCONT (reanudación), SIGTERM (terminación ordenada) y SIGKILL (terminación forzada).
- **Prioridad:** se controla mediante el valor nice (con rango de -20 a 19). ProcGuard utiliza `setpriority()` para modificar dinámicamente la prioridad de procesos problemáticos, reduciendo el acceso del proceso al CPU sin terminarlo.
- **Afinidad de CPU:** `sched_setaffinity()` permite restringir procesos a un subconjunto de cores. ProcGuard utiliza esta capacidad para acciones correctivas como aislar procesos pesados o redistribuir el balanceo de carga cuando se detecte contención en cores específicos.
- **cgroups v2:** permite agrupar procesos para crear grupos de control y aplicar límites de recursos bajo `/sys/fs/cgroup/procguard/` con límites de CPU (`cpu.max`) y memoria (`memory.max`), para confinar procesos problemáticos sin terminarlos.

4.4. Detección de comportamiento sospechoso

La detección se basa en dos mecanismos complementarios: heurísticas acumulativas que construyen perfiles de comportamiento muestra a muestra a partir de datos de procfs, y notificación push de eventos del sistema de archivos mediante la API inotify del kernel. Las heurísticas de seguridad se fundamentan en indicadores de compromiso (IoC) observables a nivel de proceso como evidencia forense, y detectan patrones de comportamiento, no malware específico, por lo que un atacante sofisticado podría evadir las heurísticas si conoce los patrones monitoreados.

5. Arquitectura del Sistema

ProcGuard se estructura en siete módulos principales con responsabilidades delimitadas, los cuales se comunican mediante estructuras de datos compartidas.

5.1. Módulo 1: Recolector de datos (Data Collector)

Responsable de la interacción y lectura directa de archivos de procfs. Para cada proceso almacena la tupla (pid, starttime) como identificador único para evitar errores por reciclaje de PIDs.

En cada ciclo de muestreo, el recolector escanea /proc/ para descubrir procesos activos. Los procesos nuevos se registran como nuevas entradas con un buffer circular vacío en el almacén de muestras. Los procesos que desaparecen se marcan como terminados. Sus datos se conservan durante un periodo de gracia de G ciclos (valor por defecto: 10 ciclos) para permitir que alertas pendientes completen su evaluación y que los snapshots forenses puedan capturar el historial reciente. Transcurrido el periodo de gracias, el buffer circular y las estructuras asociadas se liberan.

5.2. Módulo 2: Almacén de muestras (Sample Store)

Buffer circular de las últimas N muestras para cada proceso y métricas globales. Alimenta al motor de métricas para calcular deltas, al motor de alertas para evaluar persistencia temporal de alertas y a la estructura acumulativa de las heurísticas.

5.3. Módulo 3: Motor de métricas (Metrics Engine)

Calcula las métricas derivadas como: porcentaje de CPU por proceso, hilo y core; consumo de memoria residente y proporcional; tasas de I/O; tasa de tráfico de red por interfaz; conexiones activas y sockets por proceso. Transforma datos crudos en información accionable.

Mettricas monitoreables

Métrica	Identificador	Unidad
Uso de CPU del proceso	cpu_percent	Porcentaje
Memoria residente	mem_rss	Bytes
Memoria virtual	mem_vsize	Bytes
Tasa de lectura I/O	io_read_rate	Bytes/s
Tasa de escritura I/O	io_write_rate	Bytes/s
Conexiones de red	net_connections	Entero
Tasa de bytes de red	net_bytes_rate	Bytes/s
Sockets abiertos	net_sockets	Entero
Número de Hilos	thread_count	Entero
File descriptors	fd_count	Entero

5.4. Módulo 4: Motor de alertas y gobernanza (Alert & Governance Engine)

Núcleo diferenciador de ProcGuard. Evalúa las métricas contra las políticas definidas por el administrador del sistema y ejecuta acciones correctivas. Implementa tres mecanismos contra falsos positivos:

- **Persistencia temporal:** la alerta solo se activa si el umbral se supera durante N muestras consecutivas configurables.
- **Histéresis:** la desactivación de las alertas requieren que la métrica descienda por debajo de un segundo umbral inferior durante M muestras.

- **Cooldown:** tras ejecutar una acción correctiva, se espera un período de enfriamiento configurable antes de escalar.

El motor soporta escalamiento progresivo: el administrador define una secuencia ordenada de acciones con severidad creciente (warn → renice → cage → stop → kill). Antes de ejecutar cualquier acción, reválida que el (pid, starttime) sigue siendo el mismo que cuando se detectó la anomalía, previniendo actuar sobre un PID reciclado.

El avance al siguiente nivel de escalamiento ocurre cuando, tras ejecutar una acción correctiva y esperar el periodo de cooldown completo, la métrica continua superando el umbral durante el periodo de persistencia configurado. Si durante el cooldown o la persistencia posterior la métrica desciende por debajo del umbral inferior (histéresis), el nivel de escalamiento se mantiene pero no avanza, y se reactiva solo si la métrica vuelve a superar el umbral con persistencia completa.

Políticas de gobernanza

Todas las políticas de rendimiento y seguridad se definen en un archivo de configuración con formato INI extendido. Cada política comparte la misma estructura de persistencia temporal, histéresis, cooldown y escalamiento progresivo, diferenciándose en el campo type (performance o security) y en los parámetros específicos. El administrador define qué políticas activar, sus parámetros de sensibilidad y la secuencia de acciones.

Catálogo de acciones

Acción	Sintaxis	Mecanismo del kernel
Advertencia	warn	Registra alerta en syslog y TUI (sin acción sobre proceso)
Cambio de prioridad	renice:<valor>	Aplica setpriority()
Reasignación de CPU	affinity:<cores>	Aplica sched_setaffinity()
Confinamiento	cage:<cpu>:<mem>	crea cgroup v2 con límites (cpu.max, memory.max)
Suspensión	stop	Envía SIGSTOP
Terminación ordenada	term	Envía SIGTERM
Terminación forzada	kill	Envía SIGKILL

Archivo de configuración

El sistema utiliza un enfoque de dos capas: ProcGuard incluye una configuración por defecto embebida en el código con políticas conservadoras (umbrales altos, acciones suaves, solo warn sin kill), y el administrador puede proporcionar un archivo de configuración que sobrescribe selectivamente lo que desee. La configuración por defecto prefiere falsos negativos a falsos positivos, y warn a kill.

El parsing del archivo de configuración se realiza mediante inih, una librería embebida de dominio público, sin dependencias externas, que parsea el formato INI estándar con secciones y pares clave=valor.

Ejemplo de archivo de configuración:

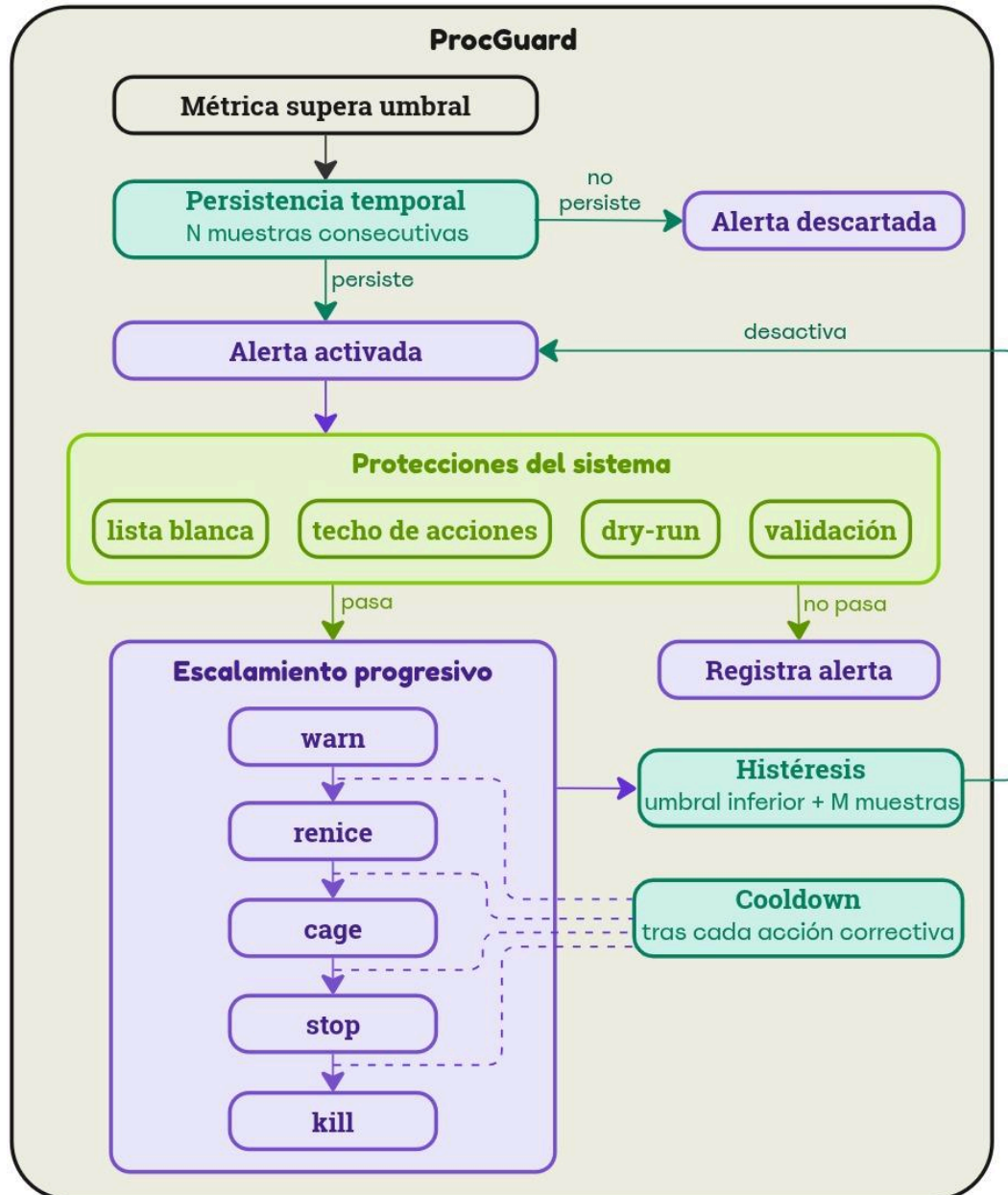
```
[policy:cpu_hog]
type = performance
metric = cpu_percent
threshold = 90
threshold_low = 70
persistence = 5
cooldown = 30
actions = warn, renice:15, affinity: 0-1, cage:50%:512M, stop, kill

[policy:port_scanner]
type = security
heuristic = port_scan
distinct_ports = 20
time_window = 10
risk_level = high
actions = warn, cage:25%:256M, stop

[global]
sample_interval = 500
net_sample_divisor = 4
sample_buffer = 120
protected_pids = 1
protected_names = systemd, sshd, procguard
max_kills_per_minute = 3
max_caged_processes = 10
dry_run = true
```

Cada sección [policy:nombre] se mapea a una estructura de política en memoria durante la carga inicial, y la sección [global] se mapea a la estructura de configuración del sistema. La validación semántica (rangos válidos de umbrales, existencia de métricas referenciadas, coherencia de secuencia de acciones) se realiza después del parsing como una etapa separada, reportando todos los errores encontrados antes de abortar el inicio si la configuración es inválida.

Diagrama de ciclo de gobernanza



5.5. Módulo 5: Motor de seguridad (Security Engine)

Módulo especializado que analiza patrones de comportamiento de los procesos para identificar actividad potencialmente maliciosa. Este motor de detección de comportamiento sospechoso opera como una etapa adicional de evaluación dentro del ciclo de gobernanza, consumiendo tanto los datos acumulativos de procfs como los eventos capturados por el hilo dedicado de inotify.

Utiliza dos mecanismos: análisis acumulativo entre muestras para heurísticas basadas en procs y notificación push de eventos del sistema de archivos mediante inotify para heurística de enumeración. No requiere módulos del kernel adicionales, inspección profunda de paquetes ni herramientas externas.

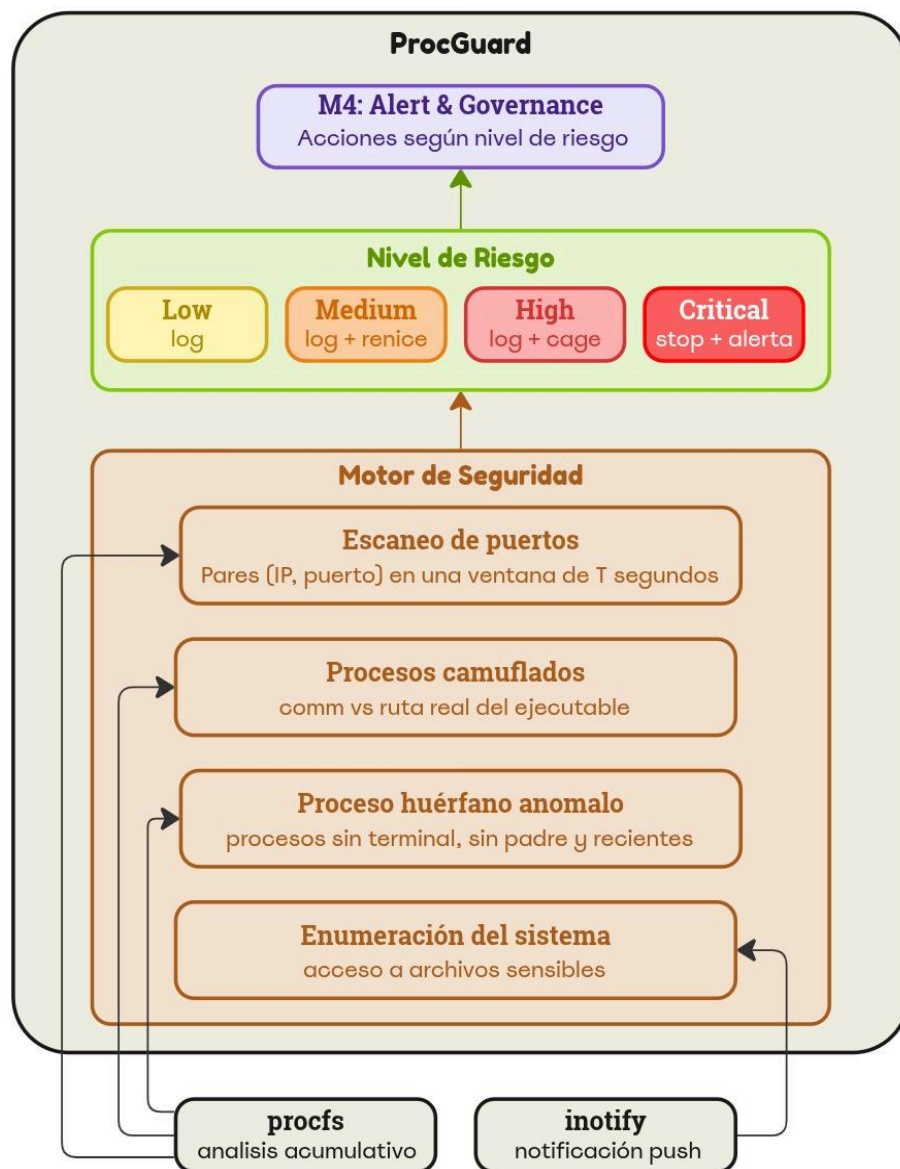
Las cuatro heurísticas de seguridad implementadas son:

- **Escaneo de puertos:** ProcGuard mantiene por proceso una estructura de datos persistente entre muestras con un conjunto de pares (IP destino, puerto destino) observados. Cada muestra agrega los pares nuevos que aparecen en `/proc/net/tcp` para ese proceso. Si en una ventana de T segundos el conjunto acumula más de N pares distintos, se activa la alerta. Los pares se expiran tras T segundos sin reaparecer. No detecta escaneos completados en menos de un intervalo de muestreo, pero detecta todo lo que tenga duración mayor al intervalo.
- **Procesos camuflados:** compara el nombre del proceso (campo comm en `/proc/[pid]/stat`) con la ruta real del ejecutable obtenida de `/proc/[pid]/exe` mediante `readlink()`. Un proceso cuyo nombre coincide o imita un binario del sistema (sshd, systemd, cron) pero cuyo ejecutable reside fuera de las rutas estándar (`/usr/bin`, `/usr/sbin`, `/bin`, `/sbin`) se marca como sospechoso.
- **Enumeración del sistema:** se utiliza `inotify_add_watch()` sobre archivos sensibles específicos (`/etc/passwd`, `/etc/shadow`, `/etc/ssh/*`, claves privadas, archivos de configuración críticos). Cuando inotify notifica un acceso, el evento se registra con timestamp en la cola de eventos. Al procesar la cola, el hilo de gobernanza recorre `/proc/*fd/` buscando procesos que mantengan abiertos descriptores a los archivos vigilados. Si se identifica el proceso, se le atribuye el evento y se acumula en su perfil de seguridad. Si el proceso ya cerró el archivo antes del procesamiento, el evento se registra como “acceso detectado a [archivo], proceso no atribuido” en el log forense.
- **Procesos huérfanos anómalos:** identifica procesos que aparecen sin terminal asociada (`tty = 0` en `/proc/[pid]/stat`), sin un padre legítimo en el árbol de procesos (`PPID = 1` pero no son servicios conocidos) y que fueron creados dentro de los últimos `orphan_max_age` segundos (defecto: 60 segundos). Procesos huérfanos que superan esta edad sin haber sido detectados como sospechosos se consideran estabilizados y dejan de evaluarse contra esta heurística. Este patrón es común en backdoors y procesos inyectados que se desvinculan de su padre original mediante double fork.

Niveles de riesgo

Nivel	Indicador TUI	Acción predeterminada
Low	Amarillo	Registro en log y reporte; sin acción automática
Medium	Naranja	Registro + reducción de prioridad
High	Rojo	Registro + confinamiento en cgroup
Critical	Rojo parpadeante	Registro + suspensión inmediata + notificación urgente

Diagrama de componentes del Motor de Seguridad



5.6. Módulo 6: Interfaz de terminal interactiva (TUI)

Implementada con ncurses, librería que forma parte del proyecto GNU utilizada para crear interfaces de usuario basadas en texto en la terminal. Proporciona información densa pero organizada, además de una experiencia interactiva completa con múltiples vistas, paneles y acciones rápidas de navegación por teclado. Opera en un hilo de ejecución dedicado, las acciones interactivas se encolan y ejecutan dentro del hilo principal para garantizar todas las validaciones y protecciones del sistema.

Layout principal

- **Barra superior (System Bar):** uptime, carga media, uso global de CPU, memoria total/usada/disponible, memoria swap y tráfico de red por interfaz.
- **Panel principal (Process Table):** tabla interactiva con columnas como PID, usuario, prioridad, nice, estado, caged, CPU%, MEM%, I/O, conexiones de red, hilos, tiempo de ejecución, indicador de seguridad y comando.
- **Panel de detalle (Detail Panel):** información extendida del proceso seleccionado con árbol de hilos, descriptores de archivo, conexiones de red, cgroup y alertas activas asociadas.
- **Dashboard de alertas:** alertas activas con nivel de escalamiento, métrica que la disparó, valor actual vs umbral, tiempo activa, acciones ejecutadas e indicador [DRY-RUN] si aplica.
- **Panel de seguridad:** vista dedicada que muestra procesos marcados como sospechosos, la heurística que los detectó, su nivel de riesgo, el detalle del patrón observado y las acciones de gobernanza aplicadas o pendientes.

Acciones interactivas

Tecla	Acción	Descripción
k / K	Matar proceso	Envía SIGTERM (k) o SIGKILL (K)
p	Pausar/reanudar	SIGSTOP o SIGCONT
n	Cambiar prioridad	Solicita nuevo valor de nice
a	Asignar afinidad	Seleccionar cores
j	Enjaular proceso	Confinamiento en cgroup

l	Liberar proceso	Saca del cgroup de confinamiento
s	Snapshot forense	Captura manual del estado completo del proceso
f	Filtrar proceso	Por usuario, nombre, PID, estado, cgroup, riesgo de seguridad
x	Panel de seguridad	Vista de procesos sospechosos

5.7. Módulo 7: Reportes y registro forense

Responsable de la escritura del log estructurado, snapshots forenses y de la generación de reportes HTML autocontenidos mediante template + JSON. Al generar un reporte, ProcGuard lee el template, inyecta el JSON como una variable JavaScript, y escribe el resultado como un único archivo HTML autocontenido.

Log de métricas (JSON lines)

Registro estructurado de cada muestra en formato JSON lines: un objeto JSON por línea, un archivo por sesión. Cada línea contiene un timestamp, tipo de registro (muestra periódica, alerta, acción de gobernanza, evento de seguridad) y el payload de datos correspondiente. Este formato permite consulta directa con herramientas estándar y parsing trivial con cJSON, la misma librería utilizada para la generación de reportes HTML. La rotación es automática por tamaño máximo configurable (default: 50 MB) o manual (SIGUSR2 al daemon). Al abrir un archivo de log para generación de reportes, se lee secuencialmente filtrando por tipo de registro y rango temporal.

Snapshots forenses

Ante cada alerta de rendimiento o de seguridad, o por solicitud manual, se captura automáticamente:

- **Identidad:** PID, nombre, comando, usuario, PPID, ruta del ejecutable real, tiempo de inicio.
- **Métricas instantáneas:** CPU%, memoria, I/O, conexiones de red, hilos.
- **Mapas de memoria:** /proc/[pid]/maps completo.
- **Descriptores de archivo:** /proc/[pid]/fd con destino de cada descriptor.

- **Conexiones de red:** todas las conexiones TCP/UDP del proceso.
- **Historial reciente:** últimas N muestras del buffer circular.

Reportes HTML

La generación de reportes utiliza una arquitectura de template + datos. El código C genera un archivo JSON (con cJSON) con toda la data del reporte incluyendo métricas, alertas, snapshots, acciones y eventos de seguridad. Este JSON se inyecta en un template HTML preexistente que contiene CSS y JavaScript de visualización, produciendo un archivo HTML autocontenido y sin dependencias externas, que se abre en cualquier navegador. El reporte HTML contiene las siguientes secciones:

- **Resumen ejecutivo:** procesos con mayor consumo de recursos, total de alertas por tipo y severidad, acciones de gobernanza ejecutadas y su efectividad, eventos de seguridad detectados y estado general del sistema.
- **Tabla de eventos:** registro cronológico de todos los incidentes (alertas de rendimiento, acciones de gobernanza ejecutadas y eventos de seguridad) con filtrado por tipo de evento, severidad y proceso.
- **Sección de seguridad:** detalle de cada evento de seguridad detectado con heurística activada, evidencia del patrón, nivel de riesgo asignado, acción tomada y snapshot forense asociado.
- **Snapshots forenses:** detalle de cada snapshot presentado en sección navegable con tablas de descriptores de archivo, tabla de conexiones de red, listado de mapas de memoria, métricas instantáneas e identidad del proceso.

5.8. Modos de Operación

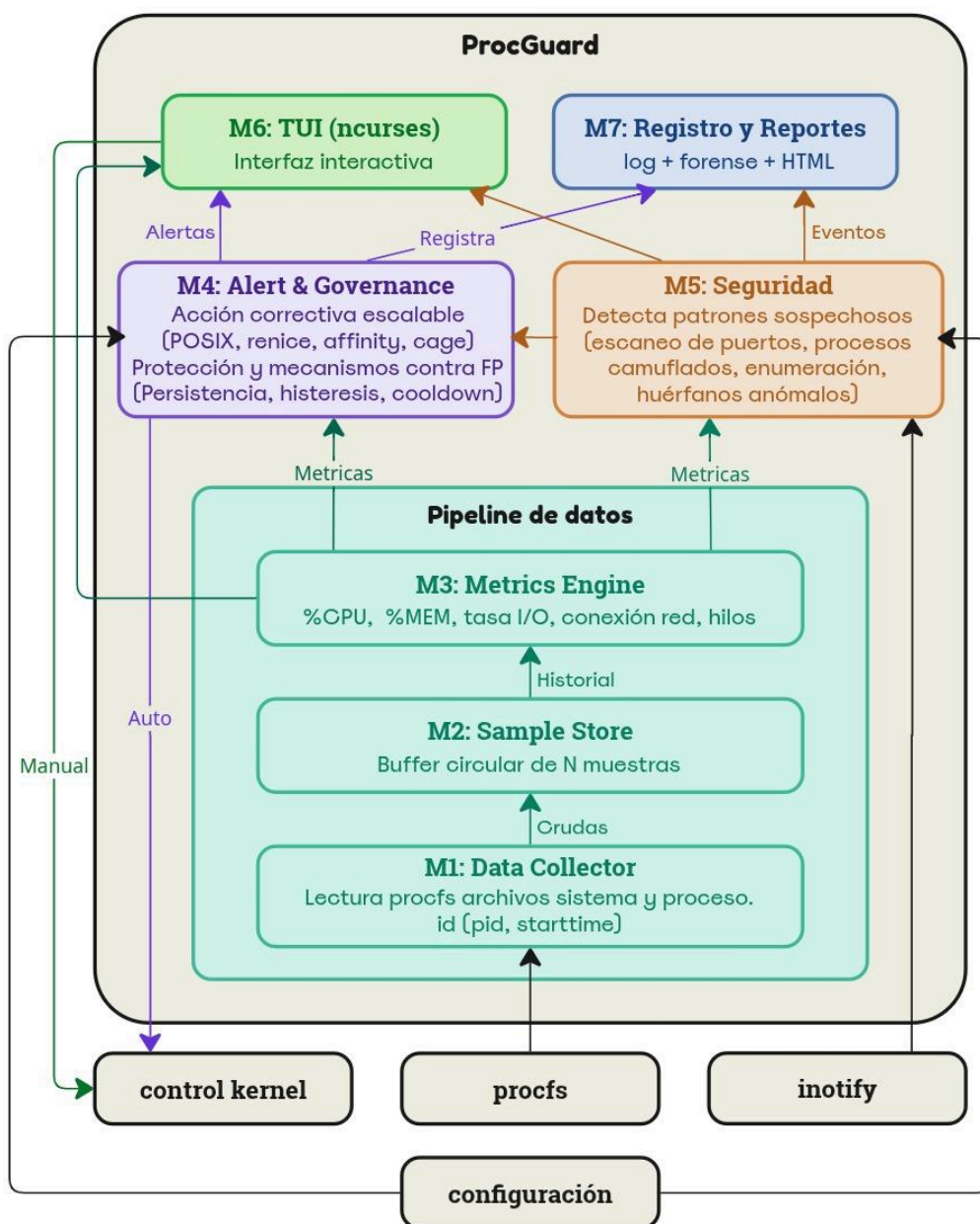
Modo interactivo (TUI)

Modo predeterminado. Lanza la interfaz de terminal completa con todos los paneles para ejecutar todo el ciclo de gobernanza, permitiendo al administrador observar, interactuar y supervisar las acciones automáticas. Incluye indicador visual [DRY-RUN] cuando el modo dry-run está activo. La TUI opera en su propio hilo de ejecución, garantizando respuesta inmediata a la entrada del usuario mientras el ciclo de gobernanza ejecuta su pipeline de forma independiente.

Modo daemon

Ejecución en segundo plano sin interfaz visual. El ciclo de gobernanza opera autónomamente; alertas y acciones se registran vía syslog. Responde a señales POSIX: SIGUSR1 para generar reportes HTML bajo demanda, SIGUSR2 para rotación del log, SIGHUP para recarga de configuración y SIGTERM para terminación ordenada con reporte final. Diseñado para ejecución continua en producción. El hilo de TUI no se instancia, el sistema opera con dos hilos: el ciclo de gobernanza y el listener de inotify.

5.9. Diagrama de Arquitectura



5.10. Ciclo de Gobernanza

El ciclo principal opera en iteraciones controladas por el intervalo de muestreo, ejecutando la siguiente secuencia:

1. **Consumir colas:** procesamiento de comandos interactivos encolados por la TUI y eventos de inotify encolados por el hilo de inotify. La cola de eventos inotify tiene un tamaño máximo configurable (default: 256 eventos). La cola de comandos interactivos de la TUI tiene un límite menor (default: 32 comandos).
2. **Recolectar:** lectura de todos los archivos de procfs relevantes con manejo resiliente de errores por proceso.
3. **Calcular:** generación de métricas derivadas a partir de muestras crudas.
4. **Evaluar rendimiento:** comparación de métricas contra políticas de rendimiento.
5. **Evaluar seguridad:** análisis de patrones de comportamiento contra heurísticas de seguridad (acumulativas e inotify).
6. **Validar:** verificación de identidad (pid, starttime) y protecciones de sistema antes de actuar.
7. **Actuar:** ejecución de acción correctiva según nivel de escalamiento (o registro en dry-run).
8. **Capturar:** generación de snapshot forense del proceso afectado.
9. **Registrar:** escritura en log y syslog.
10. **Visualizar:** depositar resultados en la estructura compartida para que el hilo de TUI los renderice (si está activo).

Cada etapa opera en modo best-effort: si una etapa falla (lectura corrupta, cálculo que produce NaN, fallo al escribir en syslog), se salta y se registra. El ciclo siguiente inicia limpio.

5.11. Concurrency

ProcGuard opera con un modelo de tres hilos de ejecución con responsabilidades delimitadas, comunicados mediante estructuras compartidas con sincronización por mutex. Este diseño responde a las tres naturalezas temporales distintas del sistema: el ciclo de gobernanza opera a intervalos regulares de muestreo, la detección de enumeración por inotify es asíncrona y orientada a eventos, y la interfaz de terminal requiere respuestas inmediata a la entrada del usuario independientemente del estado y del ritmo del ciclo de gobernanza.

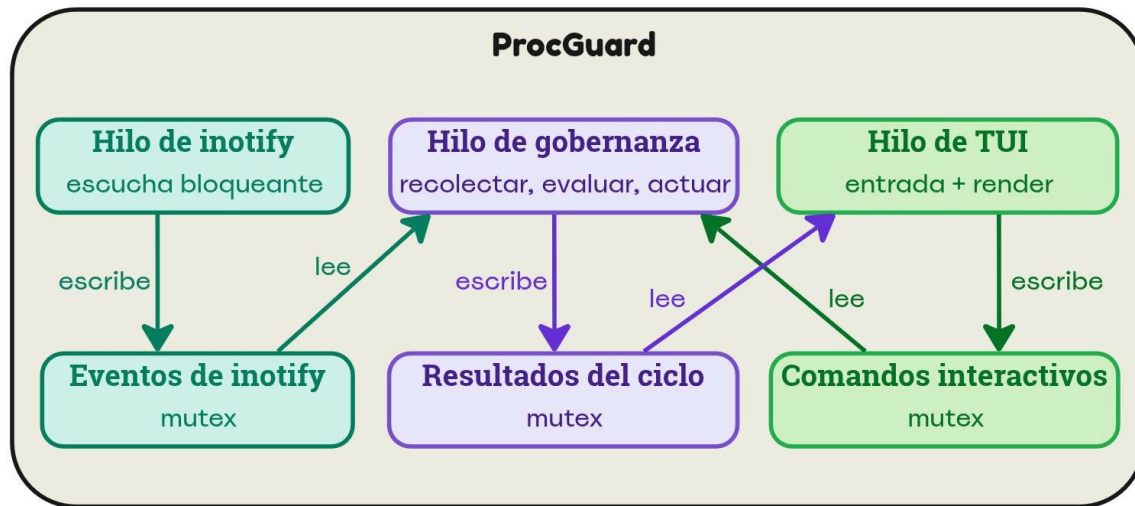
- **Hilo 1 – Ciclo de gobernanza:** ejecuta todo el pipeline completo del ciclo de gobernanza de ProcGuard en cada intervalo de muestreo, desde la recolección hasta el registro. Al inicio de cada iteración, consume los comandos interactivos encolados por el hilo de TUI (si está activo) y los eventos de inotify encolados por el hilo de inotify. Al finalizar cada iteración, deposita los resultados del ciclo en una estructura de resultados compartida. Es el único hilo que lee procfs, calcula métricas, evalúa políticas y ejecuta acciones de gobernanza.
- **Hilo 2 – Listener de inotify:** dedicado exclusivamente a la escucha bloqueante de eventos del sistema de archivos mediante `read()` sobre el descriptor de inotify. Cuando detecta un acceso a un archivo sensible vigilado, escribe el evento con su timestamp en una cola de eventos protegida por mutex. El hilo de gobernanza consume y vacía esta cola una vez por ciclo durante la fase de evaluación de seguridad.
- **Hilo 3 – Interfaz de terminal:** opera en un loop continuo con dos funciones: lee entrada del teclado mediante `getch()` con timeout de 100ms para evitar bloqueo, y redibuja la pantalla leyendo la estructura de resultados compartida. Cuando el usuario ejecuta una acción interactiva, la deposita en una cola de comandos protegida por mutex que el hilo de gobernanza procesa al inicio de la siguiente iteración. En modo daemon, este hilo no se instancia.

Sincronización

La comunicación entre hilos se realiza mediante tres estructuras protegidas individualmente con mutex de pthreads. No existen dependencias circulares entre mutexes: ningún hilo necesita adquirir más de un mutex simultáneamente, lo que elimina la posibilidad de deadlocks por diseño.

Estructura	Escritor	Lector
Resultados del ciclo	Hilo de gobernanza	Hilo de TUI
Cola de eventos inotify	Hilo de inotify	Hilo de gobernanza
Cola de comandos interactivos	Hilo de TUI	Hilo de gobernanza

Diagrama de Modelo de Concurrency



6. Manejo de Errores y Resiliencia

La estrategia de resiliencia se estructura en tres niveles que garantizan que ProcGuard continúe operando ante fallos parciales sin comprometer la integridad del sistema.

- **Nivel 1 - Lectura de procs:** toda función que lea un archivo de /proc retorna un código de estado (éxito o tipo de fallo). Los fallos ENOENT (no existe archivo/path), ESRCH (no existe proceso) y EACCES (permiso denegado) se tratan como condiciones normales y se descartan silenciosamente. Solo fallos inesperados como ENOMEM (sin memoria) e EIO (error de I/O) se registran en log. El recolector nunca aborta por un proceso individual que falla, siempre continua con el siguiente.
- **Nivel 2 - Validación antes de actuar:** antes de ejecutar cualquier acción sobre un proceso (kill, renice, cage), se reválida que el (pid, starttime) sigue siendo el mismo

que cuando se detectó la anomalía. Si el PID fue reciclado (starttime cambió), la acción se cancela y se registra como “proceso desaparecido antes de acción”. Esto previene actuar sobre el proceso equivocado.

- **Nivel 3 - Resiliencia del ciclo principal:** el ciclo de gobernanza está envuelto en manejo de errores que garantiza que un fallo en cualquier etapa individual no detiene el sistema completo.

7. Protección del Sistema

Las protecciones no limitan la detección (ProcGuard sigue detectando todo), tan solo limitan la acción automática para prevenir daño autoinfligido.

- **Lista blanca inmutable:** se define en configuración y se carga al inicio. ProcGuard nunca ejecuta acciones de gobernanza sobre PID 1, PID del propio ProcGuard y sus hijos, procesos del kernel (donde /proc/[pid]/exe apunta a vacío), y procesos que cumplan simultáneamente dos condiciones: su nombre coincide con la lista de protegidos (systemd, sshd, login, y los que el administrador defina) y su ejecutable real reside en una ruta estándar del sistema. Un proceso cuyo nombre coincide con un protegido pero cuyo ejecutable reside fuera de las rutas estándar no recibe protección y queda sujeto a la evaluación normal del motor de seguridad, que lo detectaría como proceso camuflado. La detección y alerta si se activan para procesos protegidos, simplemente no se ejecuta la acción automática. Se registra “alerta activada para proceso protegido, acción automática omitida”.
- **Techo de acciones:** máximo configurable de kills por minuto (default: 3) y de procesos confinados simultáneamente (default: 10). Si se alcanza el techo, ProcGuard entra en modo de solo alertas hasta que el administrador intervenga o el cooldown expire. Esto previene cascadas destructivas donde un falso positivo masivo lleve a matar decenas de procesos.
- **Modo dry-run:** la configuración por defecto tiene dry_run = true. El administrador debe cambiarlo explícitamente a false. En dry-run, ProcGuard ejecuta todo el ciclo completo (detecta, evalúa, decide la acción) pero en lugar de ejecutar la acción, la registra en log y la muestra en TUI con un indicador visual [DRY-RUN]. Esto

permite validar que las políticas hacen lo que se espera antes de darles poder real sobre el sistema.

- **Validación de cordura:** antes de un kill o stop, ProcGuard verifica que el proceso lleva un tiempo mínimo configurable (default: 5 segundos) en estado de alerta activa. Esto previene que un spike instantáneo (perfectamente normal en muchos workloads) active una terminación inmediata. Combinado con la persistencia temporal de N muestras, esto crea una doble barrera contra acciones precipitadas.

8. Estrategia de Testing

La estrategia de testing se planifica desde el inicio pero se implementa progresivamente junto con cada módulo. Se estructura en tres niveles complementarios.

- **Tests unitarios por módulo:** para cada módulo se escriben funciones de test que alimentan datos conocidos y verifican resultados esperados. Se implementa con un framework mínimo basado en macros assert con mensajes, sin dependencias externas. Se compila como binario aparte (make test) y se ejecuta.
- **Tests de integración con procfs sintético:** la ruta base de /proc es un parámetro configurable. Para testing se crea un directorio /tmp/fake_proc/ con la estructura de /proc: subdirectorios numéricos con archivos stat, status, io, etc., con contenido controlado. Se lanza el recolector apuntando a /tmp/fake_proc/ y se verifica que produce las métricas esperadas mediante la simulación de procesos. El pipeline completo (recolectar → calcular → evaluar → decidir acción) se prueba de forma determinista y reproducible.
- **Tests end-to-end:** scripts en Bash que lanzan ProcGuard real, inyectan una carga conocida (stress-ng), esperan, y verifican que se generó la alerta esperada y que se ejecutó la acción correcta revisando logs. Validan que todo el sistema funciona como unidad en condiciones reales de operación.

9. Tecnologías y Herramientas

Componente	Tecnología
Lenguaje principal	C (estándar C11)
TUI	ncurses / ncursesw
Control de recursos	cgroups v2 (interfaz de sistema de archivos)
Concurrencia	pthread (POSIX Threads, estandar del sistema)
Parses de configuración	inih (librería embebida, un archivo .c y .h, dominio público)
Detección de enumeración	inotify (API del kernel)
Serialización JSON para log y reportes	cJSON (un .c y un .h, sin dependencias)
Registro de eventos	syslog (interfaz POSIX estándar)
Reportes HTML	Template HTML + CSS + JavaScript con inyección de JSON
Control de Versiones	Git / GitHub
Análisis estadístico	Python (scipy, matplotlib)
Cargas de prueba	stress-ng, scripts personalizados en C/Bash
Simulación de amenazas	Scripts en C/Bash para inyección de comportamiento sospechoso
Sistema operativo	Linux (kernel 5.x+, cgroups v2)

10. Diseño Experimental

La validación de ProcGuard se estructura en dos dimensiones experimentales complementarias, con un mínimo de 12 repeticiones por tratamiento y orden aleatorizado para garantizar poder estadístico suficiente.

10.1. Dimensión 1: Funcionalidad – Precisión de detección y efectividad de gobernanza

Esta dimensión responde a la pregunta: ¿ProcGuard detecta correctamente las anomalías de rendimiento y las acciones correctivas son efectivas?. Se inyectan anomalías conocidas y se mide la capacidad del sistema para detectarlas y resolverlas.

Factores experimentales

Factor	Niveles	Justificación
Tipo de carga	CPU-bound, memory-bound	Comportamiento bajo distintos perfiles de estrés
Perfil de anomalía	Pico instantáneo, sostenido	Robustez ante falsas alarmas y efectividad ante anomalías reales
Política de gobernanza	Solo alertas, escalamiento completo	Impacto de cada nivel de acción correctiva

Variables de respuesta

Variable	Unidad	Significado
Latencia de detección	ms	Tiempo entre inicio de anomalía y activación de alerta
Tasa de verdaderos positivos	%	Anomalías correctamente detectadas / total inyectadas
Tasa de falsos positivos	%	Alertas sin anomalía real / total de alertas
Efectividad de gobernanza	%	Anomalías resueltas por acción automática / total de acciones ejecutadas

Análisis estadístico

- **Estadística descriptiva con intervalos de confianza al 95%:** para todas las variables. Los intervalos de confianza permiten afirmar con rigor que la tasa real se encuentra dentro de un rango, para no reportar solo un promedio sin incertidumbre.
- **ANOVA de una vía:** para comparar latencias entre tipos de carga y entre políticas. Responde si hay diferencias significativas.

- **Tukey HSD:** para identificar entre qué pares de tratamientos existen diferencias cuando el ANOVA es significativo. La corrección de Tukey controla la tasa de error al hacer múltiples comparaciones.
- **Verificación de supuestos:** Shapiro-Wilk para normalidad y Levene para homocedasticidad.

10.2. Dimensión 2: Seguridad – Detección de comportamiento sospechoso

Esta dimensión responde a la pregunta: ¿Las heurísticas de seguridad detectan correctamente patrones maliciosos sin generar falsas alarmas?. Se inyectan comportamientos maliciosos simulados y se mide la capacidad de detección.

Escenarios de inyección

Escenario	Implementación	Heurística evaluada
Escaneo de puertos	Script con conexiones TCP a puertos secuenciales	port_scan (acumulativa)
Proceso camuflado	Binario imitando servicio del sistema desde /tmp	disguised_process
Enumeración del sistema	Script que accede a archivos sensibles	system_enumeration (inotify)
Huérfano anómalo	Proceso desvinculado (double fork) y que opera sin terminal	orphan_anomaly
Control negativo	Servidor web, compilación, backup	No debe alertar

Variables de respuesta

Variable	Unidad	Significado
Tasa de detección por heurística	%	Inyecciones detectadas / total de inyecciones por tipo
Tasa de falsos positivos	%	Alertas ante comportamiento legítimo / total de controles
Latencia de detección de amenaza	ms	Tiempo entre inicio del comportamiento malicioso y alerta

Puntuación F1 por heurística	0–1	Media armónica de precision y recall
------------------------------	-----	--------------------------------------

Análisis estadístico

- **Matriz de confusión:** visualiza por heurística la distribución de verdaderos positivos, verdaderos negativos, falsos positivos, y falsos negativos.
- **Estadística descriptiva con intervalos de confianza al 95%:** para cada tasa, usando aproximación normal para proporciones.
- **Prueba exacta de Fisher:** para evaluar si la tasa de detección de una heurística es significativamente mejor que el azar. Relevante cuando el número de inyecciones por escenario es limitado, es decir para muestras pequeñas.

10.3. Protocolo experimental común

Cada tratamiento experimental, independientemente de la dimensión, sigue el siguiente protocolo estandarizado:

- **Preparación:** reinicio del sistema a estado limpio; verificación de que no hay procesos residuales de pruebas anteriores.
- **Línea base:** medición durante 60 segundos sin ProcGuard para establecer referencia.
- **Inicio del monitor:** lanzamiento de ProcGuard con la configuración del tratamiento.
- **Estabilización:** espera de 30 segundos para que el monitor alcance estado estable.
- **Inyección:** activación de la carga sintética o del comportamiento sospechoso.
- **Observación:** periodo de medición de 120 segundos registrando todas las variables.
- **Recolección:** terminación ordenada, extracción de reportes y logs.

10.4. Resumen de métodos estadísticos

La siguiente tabla consolida todos los métodos utilizados, su propósito específico en el experimento y la condición que determina su uso.

Método	Propósito	Cuándo se usa
Estadísticas descriptivas	Media, desviación, mediana, percentiles, IC 95%	Siempre, como primer paso
Shapiro-Wilk	Verificar distribución de normalidad de los datos	Antes de prueba paramétrica
Prueba de Levene	Verificar homocedasticidad	Antes de ANOVA
ANOVA de una vía	Comparar medias de 3+ grupos	Dimensión de funcionalidad
Tukey HSD	Identificar pares de grupos que difieren	Después de ANOVA significativo
Matriz de confusión + F1	Visualizar distribución de aciertos/errores por heurística	Dimensión de seguridad
Prueba exacta de Fisher	Evaluar si la detección es mejor que el azar	Muestras pequeñas en escenarios de seguridad

11. Cronograma

Fases de desarrollo

- **Fase 1 – Fundamentos:** diseño de arquitectura, estructuras de datos, formato de configuración (dos capas), recolector básico (CPU, memoria) con manejo resiliente de errores, y motor de métricas. Tests unitarios y cálculo de métricas.
- **Fase 2 – Recolector completo:** extensión a I/O, red (correlación socket→proceso con estructura acumulativa), hilos; buffer circular de muestras. Tests con procs sintético.
- **Fase 3 – Motor de gobernanza:** alertas con persistencia, histéresis, cooldown; acciones correctivas; protecciones del sistema. Tests unitarios de evaluación de políticas.

- **Fase 4 – Motor de seguridad:** cuatro heurísticas de detección (acumulativas e inotify); integración con políticas y niveles de riesgo. Tests con procs sintético para escenarios de seguridad.
- **Fase 5 – Interfaz TUI:** vistas del sistema incluyendo tabla de procesos, barra de sistema, panel de detalle, dashboard de alertas, panel de seguridad y acciones interactivas.
- **Fase 6 – Registro y reportes:** log, snapshots forenses, reportes HTML con template + JSON. Template HTML con CSS y JS de visualización.
- **Fase 7 – Daemon:** modo daemon con señales POSIX.
- **Fase 8 – Experimentación:** diseño experimental, ejecución, análisis estadístico y documentación.

Transversalmente a todas las fases: tests unitarios, tests de integración con procs sintético, tests end-to-end progresivos y documentación del código.

Diagrama de Gantt

